

Funzioni

Table of contents

Funzioni	2
Obiettivi	2
Il principio DRY	4
Funzioni: utilità e definizione	4
3.1 Definizione, sintassi, parametri e variabili locali	5
Chiamata della funzione	6
Esempi e definizioni	7
1 - Azione	7
2 - Azione con parametri	8
Parametri attuali e formali	8
3 - Restituzione di un valore	10
4 - Restituzione di più valori	12
Ancora sui parametri	12
Parametri e valori di default	13
Altro esempio	14
Scope di una funzione (cenni)	14
RETURN	15
Cosa fa return	16
Differenza tra print e return	16
1. Funzione senza return esplicito	17
2. Funzione con return x	18
3. Funzione con return x, y	18
4. Funzione con return x, y, z	18
Compatibilità tra numero di variabili e numero di valori	19
Scope: dove sono visibili le variabili	19
Alterare una variabile locale	21
Alterare una variabile locale con lo stesso nome di una variabile esterna	21
Idea fondamentale	22
Mutabilità e passaggio degli argomenti	22
Esempio con una lista	22
Esempio con un intero	23
Osservazione	23

Confronto: modificare un immutabile passato come parametro	24
Confronto: modificare un mutabile passato come parametro	24
Riassegnare un parametro non è la stessa cosa che modificare l'oggetto	25
Differenza tra <code>def f(a)</code> e <code>def f(a=[])</code>	26
Caso 1: parametro obbligatorio	26
Caso 2: parametro con default mutabile	27
Perché accade?	27
Soluzione corretta	27
Regola pratica	28
Funzioni che chiamano altre funzioni	29
Esempio	29
Altro esempio	29
Gestione degli errori e dei casi problematici	30
Buone pratiche nella scrittura delle funzioni	32
Errori comuni	33
ESERCIZI	34
ESEMPI - SCOPE	37

Funzioni

Obiettivi

In questa sezione ci occupiamo di:

- capire perché le funzioni sono utili;
- definire e chiamare funzioni;
- usare parametri e valori restituiti;
- distinguere parametri formali e argomenti attuali;
- comprendere variabili locali e scope;
- distinguere tra `print` e `return`;
- scrivere funzioni con più parametri;
- usare valori di default;
- far collaborare più funzioni tra loro;
- gestire semplici errori e casi problematici;
- progettare funzioni chiare, riutilizzabili e facili da correggere.

Questa sezione si collega direttamente a ciò che già conosciamo: variabili, strutture dati, costrutti condizionali e iterativi, mutabilità e immutabilità.

Nota:

Queste dispense sono ancora in una versione 'bozza'. L'impaginazione è carente e possono essere presenti ripetizioni. Sono comunque complete e corrette. Segnalatemi pure errori e inesattezze.

Consideriamo i seguenti esempi di azioni *ripetute più volte* all'interno del programma. Ipotizziamo di voler ripetere le azioni in diversi punti del codice, quindi **non è possibile risolvere il problema inserendo il codice in un ciclo**.

1. Vogliamo stampare le seguenti stringhe, sempre uguali

```
*****  
*  BENVENUTI NEL MIO PROGRAMMA  *  
*****
```

2. Dato il nome dell'utente, vogliamo stampare varie volte le seguenti stringhe, sempre uguali, a parte il nome dell'utente:

```
*****  
*  MARIO, BENVENUTO NEL MIO PROGRAMMA  *  
*****
```

3. Data una sequenza di numeri, vogliamo calcolarne la media
4. Data una sequenza di numeri interi, vogliamo costruire due liste, una con i valori pari e una con i valori dispari.

Si noti che i quattro esempi sono di complessità crescente:

1. stampare stringhe sempre uguali
2. stampare stringhe che contengono un valore
3. calcolare un valore (la media) che dipende da altri valori (la sequenza di numeri)
4. costruire due liste a partire da una lista.

Il copia-incolla **NON** è una soluzione adeguata. Il codice diventa lungo e difficile da correggere e modificare ('manutenere'), con notevoli porzioni di codice duplicato. Il codice duplicato allunga inutilmente il programma, rende più difficile trovare gli errori, obbliga a fare la stessa modifica in più punti ed aumenta il rischio di incoerenze.

Il principio DRY

DRY è l'acronimo di Don't Repeat Yourself ("Non ripeterti").

La sua regola fondamentale è:

"Every piece of knowledge must have a single representation within a system". -
[The Pragmatic Programmer - Andrew Hunt, David Thomas]

L'idea di fondo è semplice: se una certa operazione o conoscenza compare più volte nel programma, è meglio scriverla una sola volta in un punto ben definito. Le funzioni sono uno degli strumenti principali per applicare questo principio.

Perché DRY è così importante:

- Meno duplicazione = meno errori
- Se un comportamento è definito in un solo punto, correggere un bug o modificarne la logica richiede un'unica modifica, anziché tante modifiche quante siano le occorrenze duplicate.
- Codice più manutenibile
- Il rischio di dimenticare di aggiornare una copia del codice o di introdurre inconsistenze sparisce.

Funzioni: utilità e definizione

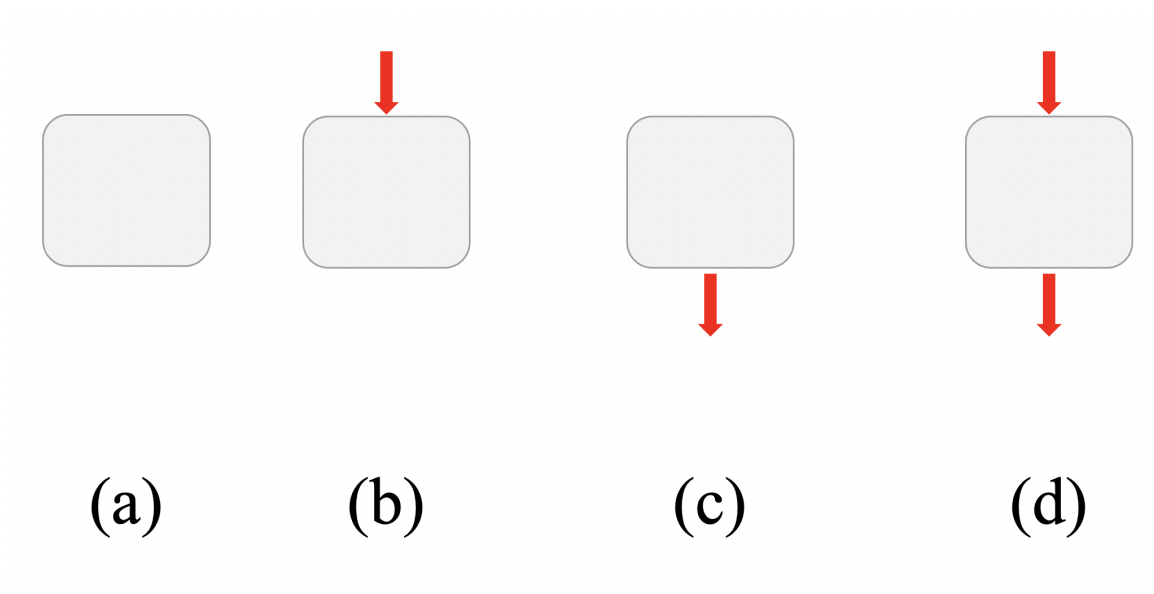
Una funzione è un **blocco di codice riutilizzabile** che svolge un'**operazione specifica**.

Le funzioni aiutano a:

- Organizzare il codice in parti logiche
- Evitare ripetizioni
- Migliorare leggibilità e chiarezza
- Agevolare la manutenzione (miglioramento, debug e modifica del codice)
- Riutilizzo e flessibilità

Una funzione può:

- non ricevere alcun dato (a, c);
- ricevere uno o più dati in ingresso; (b, d)
- non restituire un risultato (a,b)
- restituire uno o più risultati (c, d)



L'importante è che **svolga un compito chiaro**, possibilmente uno solo https://en.wikipedia.org/wiki/Single-responsibility_principle

3.1 Definizione, sintassi, parametri e variabili locali

Una **funzione** è un blocco di istruzioni identificato da un nome. Serve a raccogliere in un unico punto una sequenza di operazioni che vogliamo eseguire più volte o che vogliamo isolare perché rappresenta un compito preciso.

La sintassi generale è:

```
def nome_funzione(parametro1, parametro2, ...):  
    # corpo della funzione  
    istruzione_1  
    istruzione_2  
    return risultato
```

Gli elementi principali sono:

- **def** - parola chiave che introduce la definizione di una funzione;
- **nome_funzione** - nome scelto dal programmatore;
- parentesi tonde - contengono eventuali parametri;
- due punti - indicano l'inizio del corpo della funzione;
- indentazione - sappiamo già che l'indentazione definisce i blocchi di codice. Tutte le istruzioni indentate fanno parte della funzione
- **return** - restituisce un valore al codice chiamante.

Esempio:

```
def somma(a, b):  
    risultato = a + b  
    return risultato
```

Durante la **definizione di una funzione**, Python interpreta il blocco di codice associato all'istruzione `def`, ne verifica la correttezza sintattica e crea un oggetto funzione (**function object**) che contiene il corpo della funzione e le informazioni necessarie alla sua esecuzione.

Definire una funzione non significa eseguirla. Il codice nel corpo della funzione viene eseguito solo quando la funzione viene **chiamata**.

Successivamente, Python associa questo oggetto al nome indicato nella definizione della funzione, registrandolo nel *namespace* corrente (spazio dei nomi). Da quel momento, il nome della funzione diventa un riferimento all'oggetto funzione e **può essere utilizzato come un vero e proprio comando** richiamabile tramite parentesi, ad esempio `somma(1, 2)`.

In questo esempio:

- Python crea un oggetto funzione contenente il codice indicato
- registra il nome `somma` nel namespace corrente
- collega il nome `somma` all'oggetto funzione creato
- permette l'esecuzione del codice associato quando viene invocata `somma()`
- `a` e `b` sono **parametri formali**;
- `risultato` è una **variabile locale**;
- `return risultato` restituisce il valore calcolato.

Con `return` si interrompe l'esecuzione della funzione e si restituisce un valore. La funzione contiene sempre un `return` implicito. Se non è specificata l'istruzione `return`, la funzione restituisce l'oggetto `None`, come se terminasse con

```
def ...  
  
    return None
```

Chiamata della funzione

È importante notare che il corpo della funzione non viene eseguito durante la definizione, ma solo quando la funzione viene chiamata.

Chiamare una funzione significa richiedere a Python l'esecuzione del codice contenuto nell'oggetto funzione associato a quel nome. La chiamata avviene utilizzando il nome della funzione seguito da parentesi tonde, eventualmente contenenti argomenti.

Quando incontra una chiamata di funzione, Python:

1. cerca il nome della funzione nel namespace corrente;
2. recupera l'oggetto funzione associato a quel nome;
3. Python crea un nuovo *stack frame* (frame di esecuzione), all'interno del quale viene definito un nuovo *local scope* (spazio dei nomi locale) per la funzione. Lo *stack frame* è la struttura interna utilizzata dall'interprete per gestire una chiamata di funzione; contiene diverse informazioni di esecuzione, come le variabili locali, la posizione corrente nel codice, i riferimenti necessari all'esecuzione e l'eventuale valore di ritorno. Il *local scope*, invece, è l'insieme dei nomi locali visibili e accessibili durante l'esecuzione della funzione.
4. **assegna eventuali argomenti ai parametri della funzione** (associazione argomenti-parametri);
5. **esegue il corpo della funzione istruzione per istruzione**;
6. **restituisce il controllo al punto del programma da cui la funzione è stata chiamata, eventualmente producendo un valore di ritorno** tramite `return`.

I primi tre passaggi descrivono operazioni interne dell'interprete Python, cioè meccanismi “dietro le quinte” che preparano l'esecuzione della funzione. Gli ultimi tre passaggi, invece, riguardano ciò che accade durante l'esecuzione effettiva della funzione e producono effetti più “visibili” per il programmatore, come l'uso dei parametri, l'esecuzione delle istruzioni e l'eventuale valore restituito.

Esempi e definizioni

Ora abbiamo tutti gli elementi per risolvere i quattro problemi proposti all'inizio.

1 - Azione

Vogliamo stampare le seguenti stringhe, sempre uguali

```
*****
* BENVENUTI NEL MIO PROGRAMMA *
*****
```

Soluzione

```
# DEFINIZIONE
def ciao():
    print('*****')
    print('* BENVENUTI NEL MIO PROGRAMMA *')
    print('*****')
```

```
# Programma principale

ciao() # Chiamata
```

La funzione non accetta parametri in ingresso e non restituisce valori (in realtà restituisce None).

2 - Azione con parametri

Dato il nome dell'utente, vogliamo stampare varie volte le seguenti stringhe, sempre uguali, a parte il nome dell'utente:

```
*****
* MARIO, BENVENUTO NEL MIO PROGRAMMA *
*****
```

Soluzione

```
# DEFINIZIONE
def ciao(nome): # `nome` è il parametro FORMALE
    print('*****')
    print('* ', nome, 'BENVENUTO NEL MIO PROGRAMMA *')
    print('*****')

# Programma principale

ciao('Mario') # chiamata della funzione
# 'Mario' è il parametro ATTUALE (argomento)
```

Parametri attuali e formali

Nelle funzioni esistono due elementi distinti ma collegati:

- parametri formali
- parametri attuali (detti anche argomenti)

Questa non è una peculiarità di Python; è il meccanismo usato da tutti i linguaggi di programmazione che utilizzano le funzioni.

-
- I **parametri formali** compaiono nella definizione della funzione.


```
def ciao(nome):
```

In questo esempio `nome` è un parametro formale.

Caratteristiche:

- sono variabili dichiarate nella funzione;
- funzionano come segnaposto;
- ricevono un valore quando la funzione viene chiamata;
- esistono solo all'interno della funzione.

I parametri formali sono quindi **variabili locali**.

-
- I **parametri attuali** sono i valori passati alla funzione durante la chiamata.

```
ciao('Mario')
```

In questo caso `'Mario'` è il parametro attuale (o argomento).

Caratteristiche:

- rappresentano i dati concreti usati dalla funzione;
- vengono associati ai parametri formali;
- possono essere valori, variabili o espressioni.

Durante l'esecuzione della **chiamata**:

```
ciao('Mario')
```

il valore `'Mario'` viene assegnato al parametro formale `nome`.

All'interno della funzione, quindi:

```
nome == 'Mario'
```

La funzione userà il valore contenuto nella variabile locale `nome`.

-
- **Corrispondenza tra parametri**

Normalmente la corrispondenza tra parametri attuali e formali avviene per **posizione**.

Esempio:

```
def somma(a, b):
    print(a + b)

somma(3, 5)
```

Corrispondenza:

Parametro formale	Parametro attuale
a	3
b	5

Questa modalità si chiama passaggio posizionale dei parametri.

• Variabili locali

I parametri formali sono variabili locali della funzione. Le variabili **a**, **b** vengono create quando la funzione inizia, possono essere usate solo dentro la funzione, ed **in genere** vengono eliminate al termine della funzione (gli oggetti a cui faceva riferimento possono continuare a esistere se sono ancora referenziati altrove).

3 - Restituzione di un valore

Data una sequenza di numeri, vogliamo calcolarne la media

```
# accetta lista e rende media
def f_media(lista_valori):
    n = len(lista_valori)
    somma=0
    for el in lista_valori:
        somma = somma + el
    media =somma /n

    # Restituisce il risultato al programma chiamante
    return media

# Programma chiamante:

voti = [30, 28, 25]
m = f_media(voti)
print('media:', m)
```

- Durante la **chiamata** della funzione il parametro attuale `voti` viene copiato nel parametro formale `lista_valori`.

Più esattamente: durante la chiamata della funzione, il riferimento all'oggetto associato alla variabile `voti` viene associato al parametro formale `lista_valori`. In questo modo, all'interno della funzione, `lista_valori` e `voti` fanno riferimento alla stessa lista in memoria. **Questo dettaglio è estremamente importante per capire come gestire il passaggio di parametri in Python.**

- **Variabili locali:** `lista_valori` (che è il parametro formale), `n`, `somma`, `el`, `media`
- **Variabili globali:** Le variabili `voti` e `m` sono variabili globali, perché sono definite al di fuori della funzione. Il loro *scope* (ambito di visibilità) coincide con il namespace globale del file Python: questo significa che, dopo la loro definizione, possono essere utilizzate in qualunque punto del file (purché il codice venga eseguito dopo la loro creazione).
- Un *namespace* (spazio dei nomi) è una struttura interna usata da Python per associare i nomi agli oggetti corrispondenti. In pratica, è una “tabella” che collega identificatori come nomi di variabili o funzioni agli oggetti memorizzati in memoria.

In termini più tecnici, un file Python viene chiamato *modulo*; quindi `voti` e `m` appartengono al namespace globale del modulo.

- L'istruzione `return media` restituisce il valore contenuto nella variabile `media` al programma chiamante. Poiché la funzione viene chiamata tramite l'istruzione

```
m = f_media(voti)
```

il valore restituito dalla funzione viene assegnato alla variabile `m`. Più precisamente, analogamente a quanto avviene nel passaggio di parametri, l'istruzione `return` restituisce un riferimento all'oggetto associato a `media`, e tale riferimento viene poi associato alla variabile `m` nel programma chiamante.

Se la funzione viene chiamata semplicemente come:

```
f_media(voti)
```

il valore restituito esiste temporaneamente come risultato dell'espressione di chiamata `f_media(voti)`, ma non essendo salvato tramite assegnazione né utilizzato in un'altra espressione, il riferimento all'oggetto restituito viene scartato.

4 - Restituzione di più valori

Data una sequenza di numeri interi, vogliamo costruire due liste, una con i valori pari e una con i valori dispari.

```
def pari_dispari(lista_valori):
    lista_p=[]
    lista_d=[]
    for el in lista_valori:
        if el%2 ==0:
            lista_p.append(el)
        else:
            lista_d.append(el)

    return lista_p, lista_d

# Programma chiamante

a = [ 1,4, 6,9,11,17]
b = pari_dispari(a)
print(b)
l_p, l_d = pari_dispari(a)

print(l_p)
print(l_d)
```

Questo esempio mostra che l'istruzione return può rendere più di un valore. Tecnicamente rende una **tupla di valori**.

Ancora sui parametri

Una funzione può ricevere più di un dato.

```
def somma(a, b):
    return a + b
```

Uso:

```
x = somma(3, 5)
print(x)
```

Esempio più significativo:

```
def area Rettangolo(base, altezza):  
    return base * altezza  
  
print(area Rettangolo(4, 7))  
print(area Rettangolo(10, 3))
```

L'ordine dei parametri conta.

```
def differenza(a, b):  
    return a - b  
  
print(differenza(10, 3))  
print(differenza(3, 10))
```

I due risultati sono diversi, perché sono diversi gli argomenti passati alla funzione.

→ Esercizio

Scrivi una funzione `potenza(base, esponente)` che restituisca `base ** esponente`. Provala con almeno quattro coppie di valori.

→ Esercizio

Scrivi una funzione `massimo_di_tre(a, b, c)` che restituisca il valore massimo tra i tre numeri senza usare `max()`.

Parametri e valori di default

Una funzione può avere parametri con un **valore di default**. Questo significa che, se durante la chiamata non viene fornito un certo argomento, Python usa il valore predefinito.

```
def saluta(nome, messaggio='Ciao'):  
    print(messaggio, nome)
```

Uso:

```
saluta('Mario')  
saluta('Luca', 'Benvenuto')
```

Nel primo caso `messaggio` vale `'Ciao'`, perché non è stato specificato.

Nel secondo caso `messaggio` vale `'Benvenuto'`.

Altro esempio

```
def potenza(base, esponente=2):  
    return base ** esponente  
  
print(potenza(5))  
print(potenza(5, 3))
```

I parametri con default sono utili quando un certo valore è frequente, ma deve poter essere cambiato.

→ Esercizio

Scrivi una funzione `stampa_riga(simbolo='-', lunghezza=10)` che stampi una riga composta dallo stesso simbolo ripetuto `lunghezza` volte.

→ Esercizio

Scrivi una funzione `sconto(prezzo, percentuale=10)` che restituisca il prezzo scontato.

Scope di una funzione (cenni)

Lo scope (ambito) di una funzione è l'insieme dei nomi (variabili, funzioni, classi) a cui quella funzione può accedere in un dato punto del programma.

- Isolamento: le variabili locali non interferiscono con quelle di altri scope
- Visibilità: all'interno del corpo di una funzione, un nome viene prima cercato tra le variabili locali, poi negli scope esterni

Le variabili definite direttamente in un file `.py`, al di fuori di funzioni o classi, si chiamano **variabili a livello di modulo**.

- Le funzioni possono accedere IN LETTURA alle variabili del modulo, ma non possono alterarne il contenuto
- se una funzione crea una variabile locale omonima a quella del modulo, la variabile del modulo viene temporaneamente *mascherata* dalla variabile locale. La funzione potrà accedere alla propria variabile ma non a quelle del modulo

Attenzione, **grave errore comune**. Sfruttare la visibilità delle variabili di modulo per non passare i parametri alla funzione

```
def f_media():
    somma=0
    for el in valori:
        somma+= el
    media = somma / len(valori)
    return media

valori = [1,2,3,4,5]

m = f_media()
print(m)
```

In questo esempio, la funzione `f_media()` calcola la media correttamente solo perché legge la variabile `valori` — definita a livello di modulo — il cui nome coincide esattamente con quello usato al suo interno.

-
- Effetto collaterale nascosto: chi legge la funzione non capisce subito da dove provengano i dati.
 - Il comportamento della funzione dipende dallo stato esterno al suo *scope*.
 - La funzione non può essere riutilizzata su altre liste o tuple, vanificando la riusabilità.
 - Se un altro programma cerca di utilizzare la funzione `f_media()` ma non definisce la variabile `valori`, il codice non funzionerà
 - Se in futuro si modifica o elimina la variabile `valori`, il codice non funzionerà

Questi elementi vanificano completamente l'utilità della funzione.

La soluzione consiste sempre nel passare i dati in ingresso come parametri e restituire il risultato in uscita mediante **return**

La comprensione dello *scope* è fondamentale per l'uso corretto delle funzioni. L'argomento è dettagliato maggiormente nella parte finale di queste dispense.

RETURN

Molte funzioni servono a produrre un risultato che il programma userà più avanti.

```
def quadrato(x):
    return x * x
```

Uso:

```
q = quadrato(4)
print(q)
print(quadrato(7))
```

Cosa fa return

return:

- interrompe l'esecuzione della funzione;
- restituisce un valore al punto in cui la funzione è stata chiamata.

Esempio:

```
def esempio(x):
    if x < 0:
        return 'negativo'
    print('Questo viene eseguito solo se x non è negativo')
    return 'non negativo'
```

Se la funzione incontra **return**, termina subito.

Differenza tra print e return

Questa distinzione è fondamentale.

```
def f1():
    print(5)

def f2():
    return 5
```

Confronto:

```
x = f1()
y = f2()

print('x =', x)
print('y =', y)
```

Risultato:

- `f1()` mostra 5 sullo schermo, ma non restituisce nulla in modo utile al programma;
- `f2()` non stampa nulla, ma restituisce 5;
- di conseguenza `x` vale `None`, mentre `y` vale 5.

Regola pratica:

- si usa `print` quando vogliamo **mostrare** qualcosa all'utente;
- si usa `return` quando vogliamo **restituire** un risultato al programma.

Molto spesso, nelle funzioni che fanno calcoli, serve `return`, non `print`.

→ Esercizio

Scrivere due funzioni:

- `mostra_doppio(x)` che stampi il doppio di `x`;
- `doppio(x)` che restituisca il doppio di `x`.

Chiamale e confronta il comportamento.

1. Funzione senza `return` esplicito

Se una funzione non contiene un'istruzione `return`, Python inserisce implicitamente:

```
return None
```

La funzione restituisce quindi l'oggetto speciale `None`.

Esempi:

```
f() # il valore restituito (`None`) viene ignorato
```

```
a = f() # il riferimento all'oggetto `None` viene associato alla variabile `a`.
```

2. Funzione con return x

Se la funzione contiene:

```
return x
```

allora restituisce il valore associato a **x**.

Possibili chiamate:

```
f() # il valore restituito viene ignorato.
```

```
a = f() # il valore restituito viene associato alla variabile `a`.
```

3. Funzione con return x, y

Se la funzione contiene:

```
return x, y
```

Python restituisce la tupla (**x**, **y**)

Possibili chiamate:

```
f() # il valore restituito viene ignorato.
```

```
a = f() # La tupla `(x, y)` viene associata alla variabile `a`.
```

```
a, b = f() # tuple unpacking
```

Avviene il *tuple unpacking*:

- **a** riceve il primo valore (**x**)
- **b** riceve il secondo valore (**y**)

4. Funzione con return x, y, z

Se la funzione contiene:

```
return x, y, z
```

Python restituisce la tupla (x, y, z).

Possibili chiamate:

```
f() # La tupla restituita viene ignorata.
```

```
a = f() # la tupla `(x, y, z)` viene associata alla variabile `a`.
```

```
a, b, c = f() # tuple unpacking
```

Avviene il *tuple unpacking*:

- a = x
 - b = y
 - c = z
-

Compatibilità tra numero di variabili e numero di valori

Nel *tuple unpacking*, il numero di variabili deve coincidere con il numero di valori restituiti.

```
a, b = f()
```

Se `f()` restituisce due valori il codice è corretto. Se restituisce un numero diverso di valori, Python segnala un errore (tecnicamente, genera l'eccezione `ValueError`) perché il numero di variabili non corrisponde al numero di elementi della tupla restituita.

Scope: dove sono visibili le variabili

Lo **scope** di una variabile è la parte del programma in cui quella variabile è visibile e può essere usata.

Lo *scope* (ambito di visibilità) di una variabile indica la porzione di programma nella quale quel nome è accessibile e può essere utilizzato. In altre parole, lo scope stabilisce da quali parti del codice una variabile, una funzione o un altro identificatore può essere “visto” e referenziato.

Ad esempio:

- una variabile definita dentro una funzione ha normalmente *scope locale* e può essere usata solo all'interno della funzione;
- una variabile definita fuori dalle funzioni ha *scope globale* e può essere usata in tutto il file Python dopo la sua definizione.

In Python, una variabile creata dentro una funzione ha **scope locale**: può essere usata solo dentro quella funzione.

```
def funzione():
    x = 10
    print('Dentro:', x)

funzione()
print('Fuori:', x)    # errore
```

L'ultima istruzione genera errore, perché **x** è stata creata dentro la funzione e non esiste fuori.

Una variabile creata fuori dalle funzioni ha invece uno **scope globale** rispetto al file in cui è definita.

```
def funzione():
    print(x)

#---
x = 10
funzione()
```

Questo codice funziona perché la funzione legge una variabile definita fuori. Tuttavia, dal punto di vista didattico e progettuale, è spesso meglio passare i dati come parametri.

Meglio:

```
def funzione(x):
    print(x)

valore = 10
funzione(valore)
```

In questo modo la funzione dipende esplicitamente dai dati che riceve.

Alterare una variabile locale

Se modifichiamo una variabile locale dentro una funzione, la modifica resta dentro la funzione.

```
def modifica_locale():  
    x = 10  
    print('Prima:', x)  
    x = x + 5  
    print('Dopo:', x)  
  
modifica_locale()
```

Output:

```
Prima: 10  
Dopo: 15
```

Fuori dalla funzione, però, `x` non esiste:

```
print(x)    # errore
```

La variabile locale nasce quando la funzione viene chiamata e scompare quando la funzione termina.

Alterare una variabile locale con lo stesso nome di una variabile esterna

Consideriamo questo esempio:

```
def modifica():  
    x = 20  
    print('Dentro la funzione:', x)  
  
x = 10  
modifica()  
print('Fuori dalla funzione:', x)
```

Output:

```
Dentro la funzione: 20  
Fuori dalla funzione: 10
```

La variabile `x` dentro la funzione è locale. Anche se ha lo stesso nome della variabile esterna, non è la stessa variabile.

Questo è un punto importante: assegnare un valore a una variabile dentro una funzione crea, di norma, una variabile locale.

→ **Esercizio**

Scrivi una funzione `prova_scope()` che definisca una variabile locale `messaggio` e la stampi. Poi prova a stampare `messaggio` fuori dalla funzione e osserva l'errore.

→ **Esercizio**

Definisci una variabile globale `x = 100`. Poi scrivi una funzione che definisce al suo interno `x = 1` e stampa `x`. Verifica che fuori dalla funzione `x` valga ancora 100.

Idea fondamentale

Le funzioni lavorano in un loro spazio locale. Questo aiuta a evitare interferenze tra parti diverse del programma.

→ **Esercizio**

Scrivi una funzione `prova()` che definisca una variabile locale e la stampi. Poi, fuori dalla funzione, prova a stampare la stessa variabile e osserva cosa accade.

Mutabilità e passaggio degli argomenti

Le funzioni ricevono argomenti. Quando gli argomenti sono oggetti **mutabili**, una funzione può modificarli.

Esempio con una lista

```
def aggiungi_elemento(lista):  
    lista.append(100)  
  
numeri = [1, 2, 3]  
aggiungi_elemento(numeri)  
print(numeri)
```

Output:

```
[1, 2, 3, 100]
```

La lista originale è cambiata.

Esempio con un intero

```
def incrementa(x):  
    x = x + 1  
  
n = 5  
incrementa(n)  
print(n)
```

Output:

```
5
```

L'intero non cambia fuori dalla funzione, perché gli interi sono **immutabili**.

Osservazione

Questo tema è importante: con gli oggetti mutabili bisogna fare attenzione, perché una funzione può modificare dati usati anche altrove.

→ Esercizio

Scrivi una funzione `svuota(lista)` che elimini tutti gli elementi di una lista usando un metodo delle liste. Verifica dall'esterno che la lista originale sia stata modificata.

→ Esercizio

Scrivi una funzione `aggiungi_uno(x)` che riceva un intero, aggiunga 1 e restituisca il nuovo valore. Verifica che la variabile originale cambi solo se assegni il valore restituito.

Confronto: modificare un immutabile passato come parametro

Gli oggetti immutabili, come interi, float, stringhe e tuple, non possono essere modificati direttamente.

```
def modifica_numero(x):  
    print('Dentro, prima:', x)  
    x = x + 1  
    print('Dentro, dopo:', x)  
  
n = 10  
modifica_numero(n)  
print('Fuori:', n)
```

Output:

```
Dentro, prima: 10  
Dentro, dopo: 11  
Fuori: 10
```

Dentro la funzione, l'istruzione:

```
x = x + 1
```

non modifica l'oggetto 10. Crea un nuovo oggetto 11 e fa sì che il nome locale `x` si riferisca a quel nuovo oggetto. La variabile esterna `n` continua a riferirsi al valore originale.

Se vogliamo aggiornare `n`, dobbiamo restituire il nuovo valore e assegnarlo fuori dalla funzione:

```
def incrementa(x):  
    return x + 1  
  
n = 10  
n = incrementa(n)  
print(n)
```

Confronto: modificare un mutabile passato come parametro

Gli oggetti mutabili, come liste, dizionari e insiemi, possono essere modificati direttamente.


```
def aggiungi_elemento(lista):  
    lista.append(100)  
  
numeri = [1, 2, 3]  
aggiungi_elemento(numeri)  
print(numeri)
```

Output:

```
[1, 2, 3, 100]
```

In questo caso la funzione non crea una nuova lista. Modifica la stessa lista a cui si riferisce anche la variabile esterna `numeri`.

Riassegnare un parametro non è la stessa cosa che modificare l'oggetto

Questo esempio è diverso:

```
def sostituisci_lista(lista):  
    lista = [100, 200, 300]  
  
numeri = [1, 2, 3]  
sostituisci_lista(numeri)  
print(numeri)
```

Output:

```
[1, 2, 3]
```

Qui l'istruzione:

```
lista = [100, 200, 300]
```

non modifica la lista originale. Fa solo sì che il nome locale `lista` si riferisca a una nuova lista. La variabile esterna `numeri` resta collegata alla lista originale.

Confrontiamo:

```
def modifica_lista(lista):  
    lista.append(100)    # modifica l'oggetto originale  
  
def sostituisci_lista(lista):  
    lista = [100, 200, 300] # cambia solo il riferimento locale
```

Questa distinzione è fondamentale quando una funzione riceve oggetti mutabili.

→ **Esercizio**

Scrivi due funzioni:

- `aggiungi_zero(lista)`, che modifica la lista ricevuta aggiungendo 0;
- `nuova_lista(lista)`, che assegna al parametro una nuova lista `[0]`.

Chiama entrambe su una lista esterna e confronta i risultati.

Differenza tra `def f(a)` e `def f(a=[])`

(opzionale)

Le due definizioni sembrano simili, ma hanno un comportamento molto diverso.

Caso 1: parametro obbligatorio

```
def f(a):  
    a.append(1)  
    return a
```

In questo caso `a` è un parametro obbligatorio. Ogni volta che chiamiamo `f`, dobbiamo passare una lista.

```
lista1 = []  
lista2 = []  
  
print(f(lista1))  
print(f(lista2))
```

Output:

```
[1]  
[1]
```

Ogni chiamata lavora sulla lista che abbiamo passato esplicitamente.

Caso 2: parametro con default mutabile

```
def f(a=[]):  
    a.append(1)  
    return a
```

Ora `a` ha come valore di default una lista vuota. Il problema è che questa lista viene creata una sola volta, quando Python legge la definizione della funzione, non ogni volta che la funzione viene chiamata.

```
print(f())  
print(f())  
print(f())
```

Output:

```
[1]  
[1, 1]  
[1, 1, 1]
```

Questo risultato sorprende molti studenti: la lista di default viene riutilizzata tra chiamate successive.

Perché accade?

Il valore di default viene valutato una volta sola, al momento della definizione della funzione.

Quindi in:

```
def f(a=[]):
```

la lista `[]` non viene ricreata a ogni chiamata. È sempre la stessa lista.

Soluzione corretta

Quando serve un valore di default per una lista, si usa spesso `None`.

```
def f(a=None):  
    if a is None:  
        a = []  
    a.append(1)  
    return a
```

Ora ogni chiamata senza argomenti crea una nuova lista.

```
print(f())  
print(f())  
print(f())
```

Output:

```
[1]  
[1]  
[1]
```

Regola pratica

Evita valori di default mutabili come:

```
def f(a=[]):  
    ...
```

```
def f(dizionario={}):  
    ...
```

Preferisci:

```
def f(a=None):  
    if a is None:  
        a = []
```

Questa regola evita modifiche inattese tra chiamate diverse della funzione.

→ **Esercizio**

Esegui mentalmente questo codice e prova a prevedere l'output:

```
def aggiungi(x, lista=[]):  
    lista.append(x)  
    return lista  
  
print(aggiungi(1))  
print(aggiungi(2))  
print(aggiungi(3))
```

Poi riscrivi la funzione usando `None` come valore di default.

Funzioni che chiamano altre funzioni

Le funzioni possono collaborare. Una funzione può chiamarne un'altra.

Questo permette di suddividere un problema complesso in sottoproblemi più semplici.

Esempio

```
def quadrato(x):  
    return x * x  
  
def somma_quadrati(a, b):  
    return quadrato(a) + quadrato(b)  
  
print(somma_quadrati(3, 4))
```

Qui:

- `quadrato(x)` risolve un compito semplice;
 - `somma_quadrati(a, b)` riutilizza quella funzione.
-

Altro esempio

```
def somma(lista):  
    totale = 0  
    for el in lista:  
        totale += el  
    return totale  
  
def media(lista):  
    if len(lista) == 0:  
        return None  
    return somma(lista) / len(lista)
```

La funzione `media` sfrutta la funzione `somma`.

Vantaggi: il codice è più modulare, ogni funzione ha un compito preciso, è più facile testare le parti separatamente.

→ **Esercizio**

Scrivi una funzione `cubo(x)` e poi una funzione `somma_cubi(a, b)` che usi `cubo`.

→ **Esercizio**

Scrivi una funzione `massimo(lista)` e poi una funzione `escursione(lista)` che restituisca `massimo(lista) - minimo(lista)`, usando due funzioni separate.

Gestione degli errori e dei casi problematici

Una buona funzione non si limita a funzionare quando tutto va bene. Deve anche gestire i casi anomali più prevedibili.

Quando scriviamo una funzione non basta pensare al caso normale. Dobbiamo anche chiederci:

- cosa succede se l'input è vuoto?
- cosa succede se un valore non è valido?
- cosa succede se si rischia un errore di esecuzione?

Molti problemi si possono prevenire con un `if`.

Esempio: divisione sicura.

```
def divisione_sicura(a, b):  
    if b == 0:  
        return None  
    return a / b
```

Qui evitiamo la divisione per zero.

- `try-except` (opzionale)

Se leggiamo un intero con `int(input())`, l'utente potrebbe scrivere qualcosa che non è un numero.

```
def leggi_intero():
    testo = input('Inserisci un intero: ')
    try:
        numero = int(testo)
        return numero
    except ValueError:
        print('Errore: devi inserire un numero intero')
        return None
```

- il blocco `try` contiene istruzioni che potrebbero generare un errore;
- il blocco `except ValueError` viene eseguito se quel tipo di errore si verifica.
- Ripetere la richiesta finché l'input non è valido

```
def leggi_intero_positivo():
    while True:
        testo = input('Inserisci un intero positivo: ')
        try:
            numero = int(testo)
            if numero > 0:
                return numero
            print('Errore: il numero deve essere positivo')
        except ValueError:
            print('Errore: devi inserire un intero')
```

Questa funzione continua a chiedere l'input finché non riceve un intero positivo.

Non è necessario studiare le eccezioni. È però utile capire che Python permette di intercettare errori e reagire in modo controllato.

→ Esercizio

Scrivi una funzione `rapporto(a, b)` che restituisca a / b , ma che restituisca `None` se `b` vale 0.

→ Esercizio

Scrivi una funzione `leggi_float()` che chieda all'utente un numero reale. Se l'utente sbaglia, la funzione deve stampare un messaggio di errore e restituire `None`.

→ Esercizio

Scrivi una funzione `leggi_voto_valido()` che continui a chiedere un voto intero finché l'utente non inserisce un numero compreso tra 0 e 30.

Buone pratiche nella scrittura delle funzioni

- Scegliere nomi chiari: `def calcola_media(lista_voti):`, non `def f(x)`
- Una funzione, un compito

Una funzione dovrebbe fare una cosa precisa.

Meglio una funzione breve che calcola la media, piuttosto che una funzione lunga che:

- legge input;
- controlla errori;
- ordina dati;
- stampa risultati;
- salva su file.

-
- Evitare di mescolare troppo input, elaborazione e output

Spesso conviene separare:

- funzioni che leggono dati;
- funzioni che elaborano dati;
- funzioni che stampano dati.

Esempio:

```
def media(lista):  
    if len(lista) == 0:  
        return None  
    totale = 0  
    for el in lista:  
        totale += el  
    return totale / len(lista)
```

Questa funzione è più riutilizzabile perché non dipende da `input()` né da `print()`.

-
- Curare i casi limite

Domande utili:

- la lista potrebbe essere vuota?
- il divisore potrebbe essere zero?
- l'utente potrebbe inserire un dato sbagliato?

-
- Aggiungere una docstring nelle funzioni più importanti

```
def media(lista):  
    """Restituisce la media degli elementi della lista.  
    Se la lista è vuota, restituisce None.  
    """  
    if len(lista) == 0:  
        return None  
    totale = 0  
    for el in lista:  
        totale += el  
    return totale / len(lista)
```

La docstring aiuta a documentare il comportamento della funzione.

Errori comuni

- dimenticare `return`
- usare `print` al posto di `return`
- chiamare una funzione senza salvare il risultato

```
quadrato(5)  
# invece di  
r = quadrato(5)
```

- accedere a variabili globali invece di utilizzare il passaggio di parametri - ERRORE GRAVE

```
x = 10  
  
def stampa():  
    print(x)
```

- non considerare i casi problematici

```
def media(lista):  
    totale = 0  
    for el in lista:  
        totale += el  
    return totale / len(lista)
```

Se lista è vuota, il programma genera un errore.

ESERCIZI

```
# Funzione che restituisce un solo valore
def quadrato(x):
    return x * x

# Chiamata corretta: un valore restituito, una variabile ricevente
risultato = quadrato(4)
print("Quadrato:", risultato)

#-----

# Funzione che restituisce due valori
def quoziente_e_resto(a, b):
    quoziente = a // b
    resto = a % b
    return quoziente, resto

# Chiamata corretta: due valori restituiti, due variabili riceventi
q, r = quoziente_e_resto(10, 3)
print("Quoziente:", q)
print("Resto:", r)

# Chiamata alternativa: il valore restituito viene ricevuto come tupla
risultati = quoziente_e_resto(10, 3)
print("Risultati (tupla):", risultati)

# Funzione che restituisce tre valori

def valori(a, b):
    v1 = a+b
    v2 = a-b
    v3 = a*b
    return v1, v2, v3

# Chiamate corrette:

y1, y2, y3 = valori(4, 5)
y_tupla = valori(4, 5)
```

```
# Chiamate errate:
```

```
y1, y2 = valori(4, 5)
```

```
y1, y2, y3, y4 = valori(4, 5)
```

-
- calcola media
 - calcola mediana

La mediana è il valore che si trova nel mezzo di un insieme ordinato di numeri. Se l'insieme ha un numero dispari di elementi, la mediana è l'elemento centrale; se ha un numero pari di elementi, la mediana è la media dei due elementi centrali.

```
def calcola_mediana(lista):  
    # Ordina la lista  
    lista_ordinata = sorted(lista)  
    n = len(lista_ordinata)  
  
    i= int(n/2)  
    # Se il numero di elementi è dispari  
    if n % 2 == 1:  
  
        return lista_ordinata[i]  
    else:  
        # Se pari, media dei due valori centrali  
  
        centro1 = lista_ordinata[i-1]  
        centro2 = lista_ordinata[i]  
        return (centro1 + centro2) / 2
```

-
- calcola varianza (campionaria)

La varianza misura quanto i valori di un insieme di dati si discostano dalla media. È molto usata in statistica per misurare la dispersione dei dati.

$$S^2 = \frac{1}{n-1} \sum (x_i - \bar{x})^2$$

```

def f_media_e_varianza(valori):
    n= len(valori)
    somma=0
    for x in valori:
        somma+= x
    media = somma / n

    somma_scarti_q=0
    for x in valori:
        #somma_scarti_q = somma_scarti_q + (x- media)**2
        somma_scarti_q += (x- media)**2

    varianza_campionaria = somma_scarti_q / (n-1)

    return media, varianza_campionaria

a = [1,2,3,4,5]
b = [ 8, 4, 9, 22, 18]

m_a, v_a = f_media_e_varianza(a)
m_b, v_b= f_media_e_varianza(b)
m_c, v_c = f_media_e_varianza([-3, -6, 9, 11])

```

-
- Media pesata

```

def f_media_pesata(valori, pesi):
    n= len(valori)

    num =0
    i =0
    while i< n:
        v= valori[i]
        p = pesi[i]
        num += v*p
        i+=1

    den = 0
    for p in pesi:
        den += p

```

```

    media_pesata = num/den
    return media_pesata

# -----
voti = [18, 25, 25, 27]
crediti = [6, 6, 4,4]

mp= f_media_pesata(voti, crediti)

print('media pesata:', mp)

```

codice equivalente

```

def f_media_pesata(valori, pesi):
    num =0
    den = 0
    for v, p  in zip(valori, pesi):
        num += v*p
        den += p
    return num/den

# -----

voti = [18, 25, 25, 27]
crediti = [6, 6, 4,4]

mp= f_media_pesata(voti, crediti)

print('media pesata:', mp)

```

ESEMPI - SCOPE

```

def f():
    print('dentro:',a)
    # stampa la variabile a del modulo

```

```
# -----
# prg principale
a=5
print('fuori:',a)
f()

print('fuori:',a)
```

```
def f():
    a=10 # variabile LOCALE che maschera a del modulo
    print('dentro:',a) # vede la propria var locale

# -----
# prg principale

a=5 # variabile di modulo ('globale' a livello di modulo)
print('fuori:',a)
f()

print('fuori:',a)
```

Errore: f() cerca di usare la propria variabile locale a prima di crearla.

```
def f():

    print('dentro:',a)
    a=10 # errore
# -----
# prg principale
a=5
print('fuori:',a)
f()

print('fuori:',a)
```

All'interno di una funzione, i **parametri formali** sono trattati come **variabili locali** che vengono inizializzate con i valori degli argomenti forniti nella chiamata. Le assegnazioni fatte ai parametri formali non modificano le variabili di modulo corrispondenti (parametri attuali).

In alcuni casi, tuttavia, gli oggetti mutabili passati come argomenti possono essere modificati in-place all'interno della funzione — vedremo queste eccezioni più avanti.

```
def f(x):
    x=10

# -----
# prg principale

a=5

print('fuori dalla funzione:',a) # 5
f(a)
print('fuori dalla funzione:',a) # 5
```

Scrivere una funzione che raddoppi la variabile in ingresso

```
def raddoppia(x):
    x= x*2 # var locale!
    return x

a=10
#raddoppia(a) #
print(a) # NON FUNZIONA!
```

```
def f(x):
    x[0]=10

# -----
# prg principale
a=[1,2,3]

print('fuori:',a)
```

```
f(a)
```

```
print('fuori:',a)
```

- scrivere una funzione che date due liste, una contenente nomi di città, e l'altra le temperature, renda un dizionario con chiave: città, valore: temperatura. Le città NON possono essere duplicate nella lista.

Variante: la stessa città può comparire più volte. il valore è la LISTA di temperature per quella città.

```
def creadiz(lista_nomi, lista_valori):
    d= dict()

    for i in range(len(lista_nomi)):
        k = lista_nomi[i]
        v = lista_valori[i]
        if k in d:
            # la città è già nel dizionario;
            # d[k] è una lista
            d[k].append(v)
        else:
            # la città non è nel dizionario.
            # la inserisco e il valore è una lista con un solo elemento
            d[k] = [v]
    return d

# VARIANTE:
# for k, v in zip (lista_nomi, lista_valori):

#-----
citta=['rm', 'ca', 'rm']
temp= [27, 31, 28]
diz_temp= creadiz(citta, temp)

print(diz_temp)
```

-
- Scrivere una funzione che modifichi una lista raddoppiando il valore degli elementi di indice pari
 - Scrivere una funzione che trovi il valore massimo in una lista. Varianti: minimo; trovare sia il valore sia la posizione; se il massimo (o minimo) non è unico, trovare tutte le posizioni; trovare valori e posizioni dei primi due valori più grandi (o più piccoli)

- Scrivere una funzione che calcoli la media degli elementi di una lista. Varianti: media pesata, media robusta https://en.wikipedia.org/wiki/Truncated_mean. Si assuma che la lista contenga ALMENO 3 elementi e che gli elementi della lista siano tutti distinti. NON utilizzare le funzioni di python **min()** e **max()**
- Scrivere una funzione che, data **lista1**, costruisca **lista2** selezionando gli elementi di **lista1** in base ad un determinato criterio. Gli elementi che soddisfano il criterio vengono inseriti in **lista2**, gli altri vengono ignorati.

Esempio di criteri: il valore è pari; il valore è maggiore di una certa soglia; Il valore è maggiore di tutti gli elementi già inseriti in **lista2**; Il valore è maggiore della media di tutti gli elementi già inseriti in **lista2**; Il valore non è già presente in lista2; Il valore è già presente in lista2 al massimo k volte; Definire un criterio che agisce sugli indici invece che sui valori (es. Solo indici pari, solo indici multipli di k,...)

- Scrivere una funzione che, data una CONDIZIONE, una OPERAZIONE e una **lista1**, costruisca **lista2** inserendo gli elementi di **lista1** modificati come segue. Se la CONDIZIONE è verificata, si applica l'OPERAZIONE indicata e si inserisce l'elemento modificato. Se la condizione non è verificata, si inserisce l'elemento non modificato.

Esempio: se l'elemento è maggiore di 5 (questa è la CONDIZIONE da valutare) si inserisca in **lista2** l'elemento moltiplicato per 2 (questa è l'OPERAZIONE), altrimenti si inserisca in **lista2** l'elemento non modificato.

Su questo schema si possono definire molti esercizi utilizzando i criteri definiti negli esercizi precedenti (o inventandone nuovi) e formulando alti tipi di operazioni.

Varianti: se la CONDIZIONE non è verificata, l'elemento non viene inserito; Se è verificata la CONDIZIONE1, si esegue l'OPERAZIONE1. Se è verificata la CONDIZIONE2, si esegue l'OPERAZIONE2, ecc

Scrivere una funzione che, data **lista1**, costruisca una **lista2** con i valori crescenti CONSECUTIVI di **lista1**

```
Lista1 -> \[1 2 4 6 4 3 8 9 10 12 20\]
Lista2-> \[1 2 4 6\]
```

Varianti: Considerare solo i valori consecutivi a partire dall'indice i

Esempio:

```
i = 7
Lista1 -> \[1 2 4 6 4 3 8 9 10 12 20\]
Lista2-> \[9 10 12 20\]
```

Varianti: Considerare i valori crescenti anche se non consecutivi

Esempio:

```
Lista1 -\> \[1 2 4 6 4 3 8 9 10 12 20\]
Lista2-\> \[1 2 4 6 8 9 10 12 20\]
```

-
- Scrivere una funzione che, data una lista **lista_1**, restituisca l'indice della massima sottosequenza crescente

Suggerimento: ricavare prima due liste temporanee, una lista con gli indici di inizio delle sottosequenze crescenti e una lista con lunghezza delle sottosequenze crescenti

-
- Scrivere una funzione che, data **lista_1**, costruisca una **lista_2** i cui elementi siano la somma a due a due degli elementi di **lista_1**

Varianti:

`el_2[i] = el_1[i] + el_1[i+1]` (la **lista_2** avrà un elemento in meno di **lista_1**)

`el_2[i] = el_1[2*i] + el_1[2*i+1]` (la **lista_2** avrà la metà degli elementi di **lista_1**)

- Scrivere una funzione che, data una lista, interpreti gli indici come **la successiva posizione da esplorare**. La funzione rende il numero di iterazioni svolte e la lista di elementi esplorati. Occorre stabilire un criterio di stop

Esempio:

```
lista = [3,2,5,1,0,4 ]
```

partendo dalla posizione 0, si va alle posizioni 3 -> 1-> 2-> 5-> 4-> 0 e si ricomincia

```
lista = [3,2,5,1,100,4 ]
```

partendo dalla posizione 0, si va alle posizioni 3 -> 1-> 2-> 5-> 4-> 100 (out of bound).

Bisogna ovviamente inserire dei controlli per evitare cicli infiniti e accessi a elementi non esistenti.

Suggerimento: il nucleo base della funzione è semplicemente

```
while (...):
    i = oca\[i\]
    # ...
```

Varianti:

- dopo k passi, quante volte viene raggiunto l'elemento j?
- dopo k passi, qual è la lunghezza totale percorsa?

- definire un insieme di regole che determinano il prossimo indice e ripetere gli esercizi precedenti. Per esempio: ogni valore indica l'**incremento** dell'indice (e non la nuova posizione) `lista = [1 1 6 -3 7 -2 10 11 -3]`

partendo dalla posizione 0, si va alle posizioni 1 -> 2-> 8-> 5-> 3-> 0 e si ricomincia

- la lista contiene caratteri che codificano le scelte

`lista = ['a', 'b', 'a', 'c']`

se a, spostati in avanti di 2

se b, spostati indietro di 2

se c, spostati avanti di 1 se l'indice di provenienza è pari, altrimenti spostati indietro di 1

- ogni valore indica l'incremento dell'indice, ma la lista è circolare. Se la lista ha k elementi (tra 0 e k-1), se sono in posizione k-1 un incremento di 1 mi porta all'elemento in posizione 0.

→ **Esercizio**

Scrivi una funzione `massimo(lista)` che restituisca il massimo di una lista non vuota senza usare `max()`.

→ **Esercizio**

Scrivi una funzione `minimo(lista)` che restituisca il minimo di una lista non vuota senza usare `min()`.

→ **Esercizio**

Scrivi una funzione `escursione(lista)` che restituisca la differenza tra massimo e minimo della lista, chiamando le due funzioni precedenti.

→ **Esercizio**

Scrivi una funzione `conta_pari_dispari(lista)` che restituisca due numeri: quanti valori pari e quanti valori dispari contiene la lista.

→ **Esercizio**

Scrivi una funzione `indice_primo_zero(lista)` che restituisca la posizione del primo zero presente nella lista; se non c'è, restituisca -1.

→ **Esercizio**

Scrivi una funzione `leggi_intero_compreso(minimo, massimo)` che continui a chiedere all'utente un intero finché non ne inserisce uno compreso tra `minimo` e `massimo`.

→ **Esercizio**

Scrivi una funzione `normalizza(lista)` che restituisca una nuova lista in cui ogni elemento è diviso per la somma totale degli elementi. Se la somma vale 0, la funzione deve restituire `None`.

→ **Esercizio**

Scrivi una funzione `rimuovi_duplicati(lista)` che restituisca una nuova lista con gli stessi elementi della lista iniziale, ma senza ripetizioni e mantenendo il primo ordine di comparsa.

→ **Esercizio**

Scrivi una funzione `verifica_crescente(lista)` che restituisca `True` se la lista è ordinata in modo non decrescente, `False` altrimenti.

→ **Esercizio**

Scrivi una funzione `statistiche(lista)` che restituisca tre valori: minimo, massimo e media. Gestisci opportunamente il caso della lista vuota.